

Two-Model Ensemble IDS Guide

Complete guide for the two-model ensemble ML system with meta-learner

Table of Contents

- 1. [Overview](#)
- 2. [Architecture](#)
- 3. [Quick Start](#)
- 4. [How It Works](#)
- 5. [Using the System](#)
- 6. [Metrics & Monitoring](#)
- 7. [Configuration](#)
- 8. [Troubleshooting](#)

Overview

What is the Two-Model Ensemble?

The ensemble system combines **two ML models** with an **adaptive meta-learner** that dynamically weights predictions based on confidence. This approach:

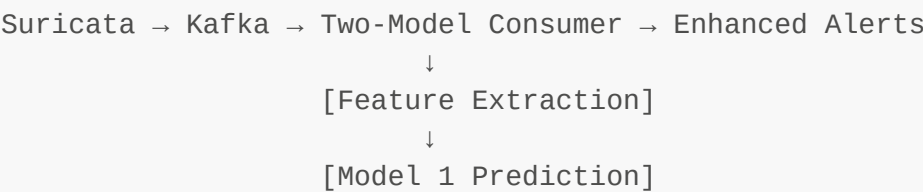
- **Reduces false positives** - Models validate each other
- **Improves accuracy** - Leverages strengths of different algorithms
- **Provides confidence metrics** - Know how reliable each prediction is
- **Adapts to traffic patterns** - Meta-learner learns optimal model weighting

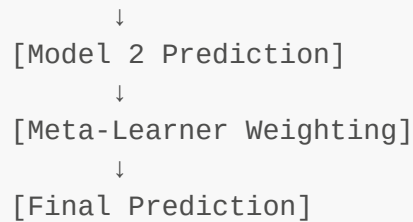
Key Components

- **Model 1 & Model 2:** Any two trained models (e.g., Random Forest, LightGBM, Logistic Regression)
- **Meta-Learner:** RandomForestRegressor that learns to weight models based on 8 confidence features
- **Feature Extractor:** Converts Suricata flow events to 34 CICIDS2017 features
- **Kafka Integration:** Consumes events from Suricata, publishes enhanced alerts

Architecture

System Flow





Components

Files:

- `two_model_ensemble.py` - Core ensemble logic with meta-learner
- `two_model_loader.py` - Model loading and initialization
- `two_model_consumer.py` - Kafka consumer with ensemble integration
- `06_start_two_model_consumer.sh` - Interactive startup script

Available Models:

- `decision_tree_model_2017.joblib/2018.joblib`
- `knn_model_2017.joblib/2018.joblib`
- `lgb_model_2017.joblib/2018.joblib`
- `lr_model_2017.joblib/2018.joblib`
- `nb_model_2017.joblib/2018.joblib`
- `random_forest_model_2017.joblib/2018.joblib`

Ensemble Methods

1. Confidence-Adaptive (Default)

- Uses pre-trained confidence weights
- Fast, no training required
- Good for immediate deployment

2. Meta-Learner (Trained)

- Collects 1000 samples, trains RandomForest to predict optimal weights
- Best accuracy after training
- Adapts to your specific traffic patterns

Quick Start

Option 1: Via Menu (Recommended)

```
sudo ./run_afpacket_mode.sh
# or
sudo ./run_dpdk_mode.sh

# Select: Option 1 → Start Complete Pipeline
```

```
# Choose: Two-Model Ensemble
# Pick your two models
# Select training option (yes/no)
```

Option 2: Manual Start

```
cd dpdk_suricata_ml_pipeline/scripts
bash 06_start_two_model_consumer.sh
```

Option 3: Direct Python

```
cd dpdk_suricata_ml_pipeline/src
python3 two_model_consumer.py random_forest_model_2017.joblib
lgb_model_2018.joblib --train
```

How It Works

Prediction Process

1. Feature Extraction

- Suricata event → 34 CICIDS2017 features
- Maps flow statistics to model input format

2. Dual Prediction

- Model 1 predicts: class + confidence
- Model 2 predicts: class + confidence
- Both run in parallel

3. Meta-Learner Weighting

- Extracts 8 confidence features:
 - Individual model confidences (2 features)
 - Max probability differences (2 features)
 - Entropy scores (2 features)
 - Agreement indicators (2 features)
- Predicts optimal weight for each model
- Combines predictions: $\text{final} = w1 * \text{pred1} + w2 * \text{pred2}$

4. Threshold Check

- If confidence $\geq$ threshold (default 0.5): Generate alert
- Otherwise: Mark as benign

Training the Meta-Learner

If enabled with `--train`:

1. Collects first 1000 flow samples
 2. Uses Suricata alerts as training labels
 3. Trains RandomForestRegressor on confidence features
 4. Switches from confidence-adaptive to meta-learner mode
 5. Continues processing with trained meta-learner
-

Using the System

Starting the Ensemble

Interactive Selection:

```
sudo ./run_afpacket_mode.sh
→ Option 1: Start Complete Pipeline
→ Choose: Two-Model Ensemble
→ Select Model 1: random_forest_model_2017.joblib
→ Select Model 2: lgb_model_2018.joblib
→ Train meta-learner? (y/N): y
```

What Happens:

- Kafka and Suricata start (if not running)
- Models load and validate
- Meta-learner initializes
- Consumer starts in background
- Returns to menu (consumer keeps running)

PROF

Viewing Logs

```
# Via menu
→ Option 9: View Logs
→ Option 2: ML consumer logs
# Automatically shows correct consumer (single or ensemble)

# Manual
tail -f dpdk_suricata_ml_pipeline/logs/ml/two_model_ensemble.log
```

Checking Status

```
# Via menu
→ Option 8: Check Status
```

```
# Shows:  
# ✓ ML Consumer (Two-Model Ensemble): Running
```

Stopping the Ensemble

```
# Via menu  
→ Option 11: Stop All Services  
  
# Or stop just consumer  
pkill -9 -f two_model_consumer.py
```

Metrics & Monitoring

What Metrics Are Tracked?

Latency Metrics:

- Feature extraction time
- Model 1 inference time
- Model 2 inference time
- Meta-learner overhead
- Total end-to-end latency
- P50, P95, P99 percentiles

Throughput Metrics:

- Events per second
- Total events processed
- Processing windows (10-second intervals)

ML Inference Metrics:

- Predictions by class (BENIGN, DDoS, PortScan, etc.)
- Confidence scores distribution
- Average inference time per prediction

Ensemble-Specific:

- Model 1 preference count (when weighted higher)
- Model 2 preference count
- Agreement rate (both models agree)
- Per-model average confidences
- Meta-learner training status

Error Tracking:

- Component errors with severity
- Error types and messages
- Timestamps for debugging

Viewing Metrics

Real-Time Dashboard:

```
./monitor_metrics.sh
```

Raw Metrics Files:

```
# JSON Lines (all metrics)
tail -f logs/metrics/metrics_YYYYMMDD.jsonl | jq '.'

# CSV files (spreadsheet-friendly)
cat logs/metrics/ml_YYYYMMDD.csv
cat logs/metrics/latency_YYYYMMDD.csv
```

Console Summary:

When you stop the consumer (Ctrl+C), you'll see:

```
┌ Performance Metrics Summary
=====
THROUGHPUT:
  Total Events: 12,450
  Avg Events/sec: 13.4

LATENCY:
  Mean: 28.45 ms
  P95: 42.18 ms
  P99: 55.67 ms

ML INFERENCE:
  Total Predictions: 12,450
  Predictions by Class:
    BENIGN: 11,987
    DDoS: 312
  Inference Latency: 26.78 ms (mean)

ERRORS:
  Total Errors: 0
=====
```

PROF

Key Metrics to Monitor

- **P95 Latency** < 50ms → Good performance
 - **Agreement Rate** > 0.7 → Models are aligned
 - **Error Count** = 0 → System healthy
 - **Throughput** matches Kafka input rate → No lag
-

Configuration

Main Config File

config/ids_config.yaml

```
kafka:
  bootstrap_servers: 'localhost:9092'
  input_topic: 'suricata-flow'
  output_topic: 'ml-alerts'
  consumer_group: 'ids-ml-consumer'

ml:
  model_dir: '../ML Models'
  prediction_threshold: 0.5
  batch_size: 1
```

Model Selection Criteria

Good Combinations:

- **Random Forest + LightGBM** - Balanced, complementary algorithms
- **Logistic Regression + Random Forest** - Simple + complex
- **Decision Tree + KNN** - Fast + context-aware

Avoid:

- Same algorithm with same training data (redundant)
- Models with very different feature expectations

Threshold Tuning

Lower threshold (0.3-0.4):

- More sensitive, catches more attacks
- Higher false positive rate

Default threshold (0.5):

- Balanced sensitivity/specificity
- Recommended for most deployments

Higher threshold (0.6-0.7):

- Only high-confidence alerts
- Lower false positives, may miss attacks

Troubleshooting

Consumer Won't Start

Check logs:

```
tail -f dpdk_suricata_ml_pipeline/logs/ml/two_model_ensemble.log
```

Common issues:

- Models not found → Check model filenames in **ML Models/** directory
- Kafka not running → Start with Option 2 first
- Config missing → Verify **config/ids_config.yaml** exists

No Predictions Being Made

Verify:

1. Suricata is running and generating flows
2. Kafka bridge is active (check Option 8)
3. Check Kafka topics have data:

```
kafka-console-consumer --bootstrap-server localhost:9092 \  
--topic suricata-flow --from-beginning --max-messages 1
```

High Latency

Check metrics:

```
./monitor_metrics.sh --status
```

Possible causes:

- System resource contention (CPU/RAM)
- Large batch sizes (check config)
- Slow model inference (try lighter models)

Models Always Disagree

Low agreement rate (<0.5):

- Models may be trained on different data distributions
- Feature mapping issues
- Consider retraining one model
- Check prediction distributions in metrics

Meta-Learner Not Training

If stuck at "Training samples: X/1000":

- Need real traffic to collect samples
- Suricata must be flagging some events as alerts
- Wait for more diverse traffic patterns
- Can skip training with `--no-train` flag

Stop All Doesn't Kill Consumer

Force kill:

```
sudo pkill -9 -f two_model_consumer.py
```

This issue was fixed - stop_all now uses `pkill -9` with verification.

Performance Characteristics

Expected Latency

Component	Typical Time
Feature Extraction	2-5 ms
Model 1 Inference	8-15 ms
Model 2 Inference	8-15 ms
Meta-Learner	1-3 ms
Total (End-to-End)	25-40 ms

PROF

Throughput

- **Single Model:** ~100-150 events/sec
- **Ensemble:** ~50-80 events/sec (2-3x slower due to dual inference)

Memory Usage

- **Single Model:** ~300-500 MB
 - **Ensemble:** ~500-800 MB (loads 2 models + meta-learner)
-

Best Practices

Model Selection

1. Choose models trained on similar data to your environment
2. Mix algorithm types (tree-based + linear, etc.)
3. Use 2017 models for general attacks, 2018 for newer patterns
4. Test combinations with `--train` to find best pairing

Training

- Enable meta-learner training for custom environments
- Let it collect 1000+ samples for good training
- Monitor agreement rate - should stabilize after training

Monitoring

- Keep metrics dashboard open during operation
- Review daily CSV exports for trends
- Set alerts for P95 latency > 50ms or error rate > 0

Production Deployment

- Use background execution (automatic via menu)
- Enable metrics logging
- Rotate log files weekly
- Archive metrics monthly for analysis

File Locations

PROF

```
IDS/
├── config/
│   └── ids_config.yaml           # Main configuration
├── ML Models/
│   ├── random_forest_model_2017.joblib # Available models
│   ├── lgb_model_2018.joblib
│   └── ...
├── dpdk_suricata_ml_pipeline/
│   ├── src/
│   │   ├── two_model_ensemble.py      # Core ensemble logic
│   │   ├── two_model_loader.py        # Model loading
│   │   ├── two_model_consumer.py      # Kafka consumer
│   │   └── metrics_logger.py          # Metrics system
│   ├── scripts/
│   │   └── 06_start_two_model_consumer.sh # Startup script
│   └── logs/
│       └── ml/
│           └── two_model_ensemble.log  # Consumer logs
└── logs/
```

```
| └─ metrics/
|     └─ metrics_YYYYMMDD.jsonl      # All metrics (JSON)
|     └─ ml_YYYYMMDD.csv            # ML metrics (CSV)
|     └─ latency_YYYYMMDD.csv       # Latency metrics (CSV)
└─ run_afpacket_mode.sh             # AF_PACKET pipeline
└─ run_dpdk_mode.sh                 # DPDK pipeline
└─ monitor_metrics.sh               # Metrics viewer
```

Additional Resources

- **Main README:** Project overview and setup instructions
- **Pipeline Architecture:** PIPELINE_ARCHITECTURE.md
- **Metrics Logger Source:** dpdk_suricata_ml_pipeline/src/metrics_logger.py
- **Example Notebooks:** notebooks/PerformanceEvaluation_AdaptiveEnsembles.ipynb

Quick Command Reference

```
# Start ensemble via menu
sudo ./run_afpacket_mode.sh → Option 1 → Two-Model Ensemble

# View ensemble logs
tail -f dpdk_suricata_ml_pipeline/logs/ml/two_model_ensemble.log

# Monitor metrics
./monitor_metrics.sh

# Check status
sudo ./run_afpacket_mode.sh → Option 8

# Stop all
sudo ./run_afpacket_mode.sh → Option 11

# Force kill consumer
sudo pkill -9 -f two_model_consumer.py

# View metrics files
ls -lh logs/metrics/
```

Last Updated: October 26, 2025

Version: 2.0 (Ensemble System)